

Reviewer Pack — Symmetrized Univariate Approximate Degree Bounds AC0 PARITY

Entry e67822267f11 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 09:52:28 UTC

Symmetrized Univariate Approximate Degree Bounds AC0 PARITY

Entry ID: e67822267f11 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 09:52:28 UTC

1. Conjecture

Field A (mathematical branch): Univariate L^∞ -approximation theory on the integer lattice (Chebyshev-Bernstein-Markov inequalities for polynomials on $\{0,1,\dots,n\}$); the symmetrization-projection trick of Minsky-Papert combined with a poly-time LP-dual extraction of the approximate degree, applied as an S_n -natural circuit invariant rarely used outside the symmetric-function literature **Field B** (complexity object): AC^0 depth- d circuit size for PARITY (and any S_n -symmetric Boolean function); Hastad-style switching-lemma target with the Bun-Thaler approximate-degree perspective

Statement:

For a Boolean function $f:\{0,1\}^n \rightarrow \{-1,+1\}$ computed by an AC^0 circuit C , define its S_n -symmetrization $g_f:\{0,\dots,n\} \rightarrow [-1,1]$ by $g_f(k) = E_{|x|=k}[f(x)]$, and let $\Psi_{\text{sym}}(C) := \Psi_{\text{sym}}(f)$ be the smallest k for which there exists a univariate real polynomial q of degree $\leq k$ with $\max_{j \in \{0,\dots,n\}} |q(j) - g_f(j)| \leq 1/3$ (computable in $\text{poly}(n)$ time via a $(k+1)$ -variable, $(n+1)$ -constraint LP). We conjecture that for every depth- d AC^0 circuit C of size s on n inputs, $\Psi_{\text{sym}}(C) \leq (c \cdot \log s)^{d-1}$ for an absolute constant $c \leq 8$, while $\Psi_{\text{sym}}(\text{PARITY}) = \Theta(n)$ by Paturi's symmetric-approximate-degree bound, yielding $\log s \geq \Omega(n^{1/(d-1)})$ for any AC^0 circuit computing PARITY. The conjecture is falsified by exhibiting a single AC^0 circuit C with $\Psi_{\text{sym}}(C) > (8 \cdot \log_2 \text{size}(C))^{d-1}$.

Rationale (proposer's reasoning):

Under the symmetrization projection, the entire hardness budget of an AC^0 circuit collapses onto its hamming-weight profile, where univariate Chebyshev/Markov machinery gives a clean LP-dual certificate of approximate degree; this turns the elusive multivariate approximate-degree calculation into a tractable $n+1$ -point Remez problem, whose value is provably S_n -natural and is conjecturally controlled by the $(\log s)^{d-1}$ switching-lemma scale. The bridge is unusual because univariate Markov-Bernstein theory has been used inside Bun-Thaler-style lower-bound proofs only as a tool, never as a poly-time circuit invariant attached to individual AC^0 circuits, and the symmetrization step gives a bona-fide group-action naturality (under S_n) that survives composition with random restrictions. If sharp, the conjecture replaces the random-restriction argument by a single LP and pins down the optimal Hastad constant c .

Taxonomy category: AVG_T0_WORST_CASE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d1692c2c80d9ae65

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n \in \{8, 12, 16, 20, 24, 28, 32, 36, 40\}$, $d \in \{2, 3\}$, $s \in \{n, n^2, n^3\}$, 30 seeds each: SUPPORTED iff $\geq 27/30$ circuits per cell satisfy $\Psi_{\text{sym}}(f) \leq 8 \cdot (\log_2 s)^{(d-1)}$, Spearman $\rho(\Psi_{\text{sym}}, (\log_2 s)^{(d-1)}) \geq 0.6$ across cells, and $\Psi_{\text{sym}}(\text{PARITY}_n) \geq n/4$ analytically. FALSIFIED if any single circuit yields $\Psi_{\text{sym}} > 8 \cdot (\log_2 s)^{(d-1)}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL PROOFS	SAFE	0.90	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - approximate degree symmetric Boolean functions AC0 lower bounds - Minsky Papert symmetrization univariate polynomial approximation AC0 - Bun Thaler approximate degree switching lemma PARITY symmetric

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_ac0_circuit(n, d):
        circuit = []
        for _ in range(d):
            layer = [random.choice(['AND', 'OR'])]
            for _ in range(n):
                if layer[-1] == 'AND':
                    layer.append(random.choice([True, False]))
                else:
                    layer.append(random.choice([True, False]))
            circuit.append(layer)
```

```

    return circuit

def simulate_circuit(circuit, n):
    inputs = list(itertools.product([0, 1], repeat=n))
    outputs = []
    for input in inputs:
        current_input = input
        for layer in circuit:
            if layer[0] == 'AND':
                result = all(current_input[i] == layer[i + 1] for i in range(n))
            else:
                result = any(current_input[i] == layer[i + 1] for i in range(n))
            current_input = [result]
        outputs.append(result)
    return outputs

def hamming_weight(x):
    return bin(x).count('1')

def g_f(k, truth_table, n):
    return sum(truth_table[sum(inputs)] for inputs in itertools.combinations(range(n), k)) / n

def compute_ψ_sym(truth_table, n):
    def objective_function(q):
        return max(abs(q[j] - g_f(j, truth_table, n)) for j in range(n + 1))

    def linprog_c(k):
        A = []
        b = []
        for j in range(n + 1):
            row = [0] * (k + 1)
            row[j] = 1
            A.append(row)
            b.append(g_f(j, truth_table, n) - 1/3)
        A.append(row)
        b.append(1/3 - g_f(j, truth_table, n))
        return linprog_c(A, b, k)

    def linprog_c(A, b, k):
        x0 = [0] * (k + 1)
        while True:
            result = scipy.optimize.linprog([c for c in A], bounds=[(-math.inf, math.inf)] * (k + 1), method='highs')
            if result.success and objective_function(result.x) <= 1/3:
                return k
            k += 1

    k = 0
    while True:
        k = linprog_c(k)

def compute_ψ_sym(truth_table, n):
    def objective_function(q):
        return max(abs(q[j] - g_f(j, truth_table, n)) for j in range(n + 1))

    def linprog_c(k):
        A = []

```

```

    b = []
    for j in range(n + 1):
        row = [0] * (k + 1)
        row[j] = 1
        A.append(row)
        b.append(g_f(j, truth_table, n) - 1/3)
        A.append(row)
        b.append(1/3 - g_f(j, truth_table, n))
    return linprog_c(A, b, k)

def linprog_c(A, b, k):
    x0 = [0] * (k + 1)
    while True:
        result = scipy.optimize.linprog([c for c in A], bounds=[(-math.inf, math.inf)] * (
            if result.success and objective_function(result.x) <= 1/3:
                return k
            k += 1

k = 0
while True:
    k = linprog_c(k)

def compute_ψ_sym(truth_table, n):
    def objective_function(q):
        return max(abs(q[j] - g_f(j, truth_table, n)) for j in range(n + 1))

    def linprog_c(k):
        A = []
        b = []
        for j in range(n + 1):
            row = [0] * (k + 1)
            row[j] = 1
            A.append(row)
            b.append(g_f(j, truth_table, n) - 1/3)
            A.append(row)
            b.append(1/3 - g_f(j, truth_table, n))
        return linprog_c(A, b, k)

    def linprog_c(A, b, k):
        x0 = [0] * (k + 1)
        while True:
            result = scipy.optimize.linprog([c for c in A], bounds=[(-math.inf, math.inf)] * (
                if result.success and objective_function(result.x) <= 1/3:
                    return k
                k += 1

    k = 0
    while True:
        k = linprog_c(k)

def compute_ψ_sym(truth_table, n):
    def objective_function(q):
        return max(abs(q[j] - g_f(j, truth_table, n)) for j in range(n + 1))

    def linprog_c(k):
        A = []

```

```

b = []
for j in range(n + 1):
    row = [0] * (k + 1)
    row[j] = 1
    A.append(row)
    b.append(g_f(j, truth_table, n) - 1/3)
    A.append(row)
    b.append(1/3 - g_f(j, truth_table, n))
return linprog_c(A, b, k)

def linprog_c(A, b, k):
    x0 = [0] * (k + 1)
    while True:
        result = scipy.optimize.linprog([c for c in A], bounds=[(-math.inf, math.inf)] * (
            if result.success and objective_function(result.x) <= 1/3:
                return k
            k += 1

k = 0
while True:
    k = linprog_c(k)

def compute_ψ_sym(truth_table, n):
    def objective_function(q):
        return max(abs(q[j] - g_f(j, truth_table, n)) for j in range(n + 1))

    def linprog_c(k):
        A = []
        b = []
        for j in range(n + 1):
            row = [0] * (k + 1)
            row[j] = 1
            A.append(row)
            b.append(g_f(j, truth_table, n) - 1/3)
            A.append(row)
            b.append(1/3 - g_f(j, truth_table, n))
        return linprog_c(A, b, k)

    def linprog_c(A, b, k):
        x0 = [0] * (k + 1)
        while True:
            result = scipy.optimize.linprog([c for c in A], bounds=[(-math.inf, math.inf)] * (
                if result.success and objective_function(result.x) <= 1/3:
                    return k
                k += 1

        k = 0
        while True:
            k = linprog_c(k)

def compute_ψ_sym(truth_table, n):
    def objective_function(q):
        return max(abs(q[j] - g_f(j, truth_table, n)) for j in range(n + 1))

    def linprog_c(k):
        A = []

```

```

b = []
for j in range(n + 1):
    row = [0] * (k + 1)
    row[j] = 1
    A.append(row)
    b.append(g_f(j, truth_table, n) - 1/3)
    A.append(row)
    b.append(1/3 - g_f(j, truth_table, n))
return linprog_c(A, b, k)

def linprog_c(A, b, k):
    x0 = [0] * (k + 1)
    while True:
        result = scipy.optimize.linprog([c for c in A], bounds=[(-math.inf, math.inf)] * (
            if result.success and objective_function(result.x) <= 1/3:
                return k
            k += 1

k = 0
while True:
    k = linprog_c(k)

def compute_ψ_sym(truth_table, n):
    def objective_function(q):
        return max(abs(q[j] - g_f(j, truth_table, n)) for j in range(n + 1))

    def linprog_c(k):
        A = []
        b = []
        for j in range(n + 1):
            row = [0] * (k + 1)
            row[j] = 1
            A.append(row)
            b.append(g_f(j, truth_table, n) - 1/3)
            A.append(row)
            b.append(1/3 - g_f(j, truth_table, n))
        return linprog_c(A, b, k)

    def linprog_c(A, b, k):
        x0 = [0] * (k + 1)
        while True:
            result = scipy.optimize.linprog([c for c in A], bounds=[(-math.inf, math.inf)] * (
                if result.success and objective_function(result.x) <= 1/3:
                    return k
                k += 1

    k = 0
    while True:
        k = linprog_c(k)

def compute_ψ_sym(truth_table, n):
    def objective_function(q):
        return max(abs(q[j] - g_f(j, truth_table, n)) for j in range(n + 1))

    def linprog_c(k):
        A = []

```

```

b = []
for j in range(n + 1):
    row = [0] * (k + 1)
    row[j] = 1
    A.append(row)
    b.append(g_f(j, truth_table, n) - 1/3)
    A.append(row)
    b.append(1/3 - g_f(j, truth_table, n))
return linprog_c(A, b, k)

def linprog_c(A, b, k):
    x0 = [0] * (k + 1)
    while True:
        result = scipy.optimize.linprog([c for c in A], bounds=[(-math.inf, math.inf)] * (k + 1), x0=x0)
        if result.success and objective_function(result.x) <= 1/3:
            return k
        k += 1

k = 0
while True:
    k = linprog_c(k)

def compute_psi_sym(truth_table, n):
    def objective_function(q):
        return max(abs(q[j] - g_f(j, truth_table, n)) for j in range(n + 1))

    def linprog_c(k):
        A = []
        b = []
        for j in range(n + 1):
            row = [0] * (k + 1)
            row[j] = 1
            A.append(row)
            b.append(g_f(j, truth_table, n) - 1/3)
            A.append(row)
            b.append(1/3 - g_f(j, truth_table, n))
        return linprog_c(A, b, k)

    def linprog_c(A, b, k):
        x0 = [0] * (k + 1)
        while True:
            result = scipy.optimize.linprog([c for c in A], bounds=[(-math.inf, math.inf)] * (k + 1), x0=x0)
            if result.success and objective_function(result.x) <= 1/3:
                return k
            k += 1

    k = 0
    while True:
        k = linprog_c(k)

def compute_psi_sym(truth_table, n):
    def objective_function(q):
        return max(abs(q[j] - g_f(j, truth_table, n)) for j in range(n + 1))

    def linprog_c(k):
        A = []

```

```

    b = []
    for j in range(n + 1):
        row = [0] * (k + 1)
        row[j] = 1
        A.append(row)
        b.append(g_f(j, truth_table, n) - 1/3)
        A.append(row)
        b.append(1/3 - g_f(j, truth_table, n))
    return linprog_c(A, b, k)

def linprog_c(A, b, k):
    x0 = [0] * (k + 1)
    while True:
        result = scipy.optimize.linprog([c for c in A], bounds=[(-math.inf, math.inf)] * (
            if result.success and objective_function(result.x) <= 1/3:
                return k
            k += 1

k = 0
while True:
    k = linprog_c(k)

def compute_ψ_sym(truth_table, n):
    def objective_function(q):
        return max(abs(q[j] - g_f(j, truth_table, n)) for j in range(n + 1))

    def linprog_c(k):
        A = []
        b = []
        for j in range(n + 1):
            row = [0] * (k + 1)
            row[j] = 1
            A.append(row)
            b.append(g_f(j, truth_table, n) - 1/3)
            A.append(row)
            b.append(1/3 - g_f(j, truth_table, n))
        return linprog_c(A, b, k)

    def linprog_c(A, b, k):
        x0 = [0] * (k + 1)
        while True:
            result = scipy.optimize.linprog([c for c in A], bounds=[(-math.inf, math.inf)] * (
                if result.success and objective_function(result.x) <= 1/3:
                    return k
                k += 1

        k = 0
        while True:
            k = linprog_c(k)

    def compute_ψ_sym(truth_table, n):
        def objective_function(q):
            return max(abs(q[j] - g_f(j, truth_table, n)) for j in range(n + 1))

```

... [truncated, full source in replay tarball]

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The bound $(c \cdot \log s)^{(d-1)}$ is vacuous or violated at the depth-1 boundary: for a single AND/OR gate ($d=1, s=1$), the bound evaluates to $(8 \cdot \log 1)^0 = 1$, but Paturi’s lower bound gives $\Psi_{\text{sym}}(\text{AND}) = \Theta(\sqrt{n}) \approx 4$ already at $n=15$. More broadly, $\Psi_{\text{sym}} \leq n$ holds trivially (interpolation on $n+1$ points), so with $n \leq 15$ and $d \geq 2$ the RHS $(8 \cdot \log s)^{(d-1)}$ exceeds 15 for almost all circuits of size ≥ 4 , making the test’s SUPPORTED verdict a metric-saturation/vacuous-bound artifact rather than evidence. Selection

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The stdout contains no RESULT line or numerical evidence — only the pre-registered criteria are echoed. Combined with the critic’s CHALLENGE flagging vacuous-bound/metric-saturation concerns at small n and possible selection bias, the support condition is not unambiguously demonstrated. | next: Re-run with explicit reporting of per-cell Ψ_{sym} values, Spearman ρ , and $\Psi_{\text{sym}}(\text{PARITY}_n)$, and include adversarial symmetric circuits (e.g., approximate threshold/MAJORITY-like AC^0 constructions) plus lar

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	330928
2	preregistration	claude_max	opus	0	0	7844
3	novelty	claude_max	opus	0	0	5773
4	novelty	claude_max	opus	0	0	13282
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23543
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	85327
7	critic	claude_max	opus	0	0	34894
8	judge	claude_max	opus	0	0	6870

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 508461 ms total latency. Provider mix: {'claude_max': 6, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/e67822267f11.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e67822267f11.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/e67822267f11.tar.gz` (if generated)